

Wise Audio Player

Simple audio framework that replaces the traditional Unity AudioSource component with a similar one that offers more features and customization.

GitHub | Asset Store | OpenUPM

Why

The Unity AudioSource component is very limited, and when you want to change a sound you have to hunt down all the prefabs and scenes that reference the old one. To remedy this, Unity added the AudioRandomContainer, which acts as an audio proxy asset that can hold a list of sounds played in a given pattern. Sadly, the UI is terrible and it doesn't give you much flexibility.

You can find a lot of audio manager and toolkit alternatives that solve these issues by placing all sound definitions in a centralized database (prefab or scriptable object) and then playing them by name string. This approach has some disadvantages:

- Having all sound definitions in one place causes conflicts when collaborating
- All sounds get loaded into memory along with the database (prefab or scriptable object)
- It needs a custom editor UI to browse and manage the sounds list, which is usually clunky
- Using strings to refer to audio assets goes against the Unity way of doing things

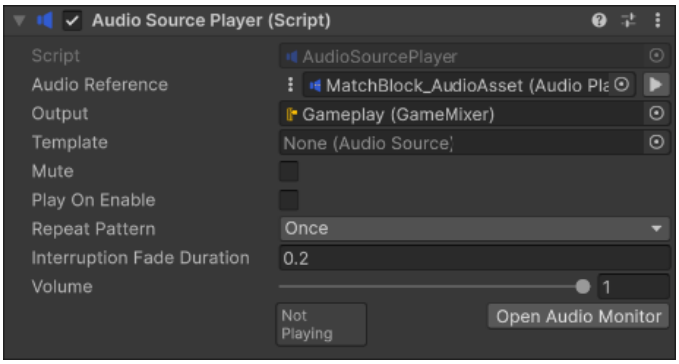
To avoid these disadvantages, an audio framework needs to decentralize the sound definitions into separate asset files and just use references to them directly. This is exactly what **Wise Audio Player** does.

Studios often write their own wrapper around AudioSource like this one, reinventing the wheel. Now you can just use this plugin instead.

Usage

This audio framework revolves around the `AudioSourcePlayer` component and the `AudioPlayerAsset` scriptable object.

AudioSourcePlayer



Replace all your Unity AudioSource components with AudioSourcePlayer, which gives you a similar interface to use. One important difference is that you can specify an AudioPlayerAsset instead of an AudioClip to use.

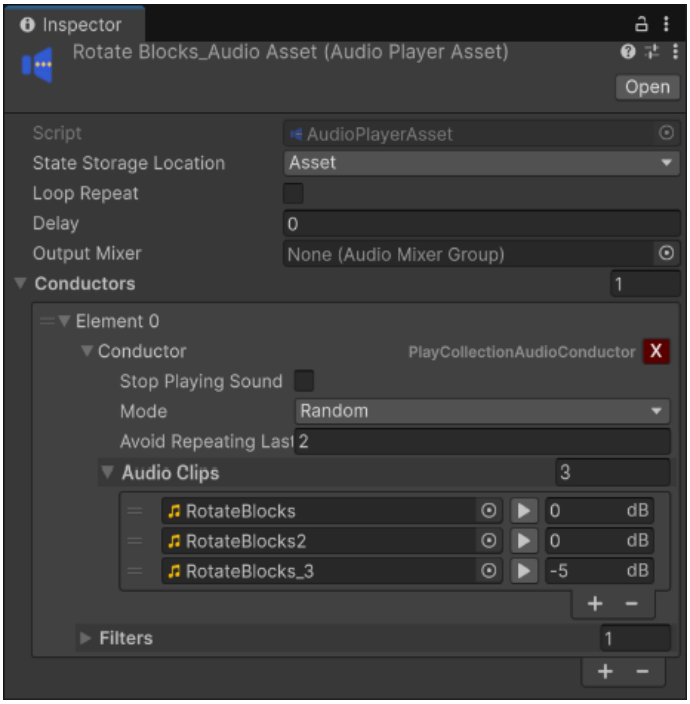
AudioSourcePlayer doesn't include all of the AudioSource's details and curves. Instead, you can specify an AudioSource template prefab to copy the details from, giving you an easy way to reuse them.

A player can play multiple sounds at the same time, each on its own AudioSource. "Sources Pool Mode" sets where these sources are created: on the player object, on a child object, or in a global pool. Sounds on the player object or its child stop immediately when it is destroyed, while sounds in the global pool are allowed to finish (looping sounds are still stopped).

"Interrupt Fade Duration" is used when the sound is manually stopped in any way, to give it a nice fade out. Even so, sounds will still stop immediately when the game object or component is destroyed.

Make sure all your sounds are played through the AudioSourcePlayer component or API so they're visible in the Audio Monitor debugging tool.

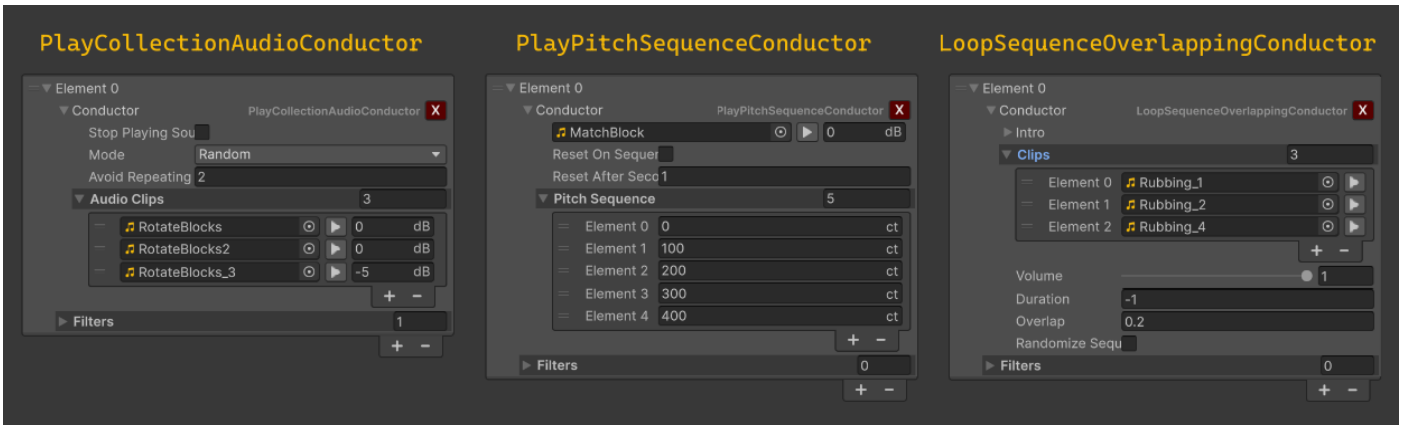
AudioPlayerAsset

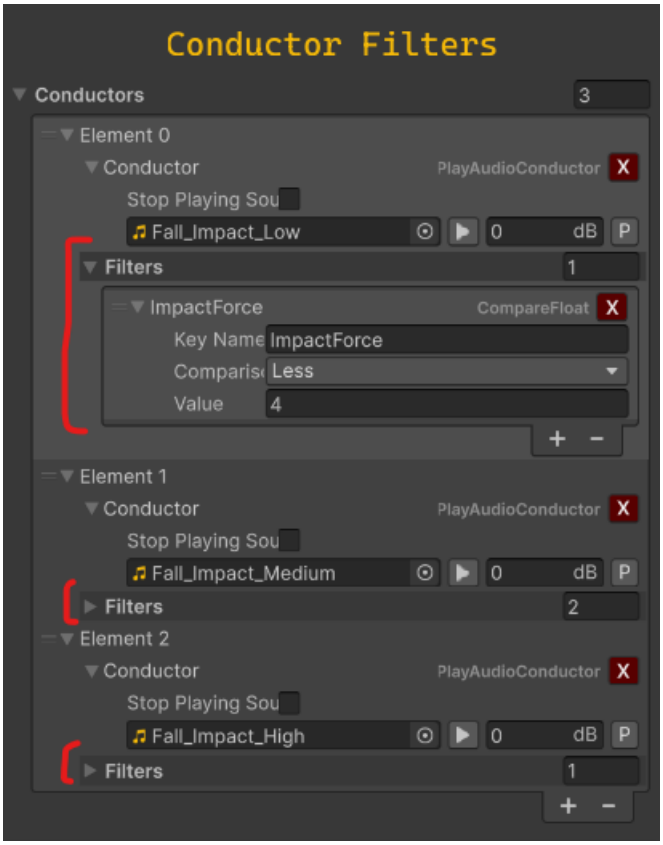


The AudioPlayerAsset serves as an audio proxy that lets you easily change what sounds should be played. Having sound definitions spread out across separate asset files makes collaborating much easier, and managing them is done through the well-known Unity Project window interface. Your programmers can link the asset in the right prefab, while your sound designer can tweak the asset itself without knowing exactly where it's played from.

The asset offers you a list of Conductors. Each conductor has conditions that check whether it should be played, and only the first one whose conditions all pass is used. Conductors decide what to play and how, once the player is triggered. The framework comes with conductors that cover most use cases, but if you need custom behaviour you can always extend them and make your own conductor type. Check the sample scene showcasing all conductor types. Here are the most notable conductor types:

- PlayCollectionAudioConductor - plays a sound picked from a list. Can play them in sequential, shuffle, or random order.
- PlayPitchSequenceConductor - plays the same sound every time, but changes the pitch according to your settings.
- IntroThenLoopConductor - plays an intro sound then loops another. Useful for music.
- LoopSequenceOverlappingConductor - loops sounds from a list by overlapping them (starts playing the next one before the last one finishes).





Some conductors need to persist their state in order to work correctly. Example: `PlayPitchSequenceConductor` needs to store which pitch was last used. These conductors have a "State Scope" setting that decides whether this state is kept per audio player (each player has its own) or per asset (shared by all players). For example, should the pitch go up no matter which barrel you hit, or should each barrel (player) track its own pitch sequence? Each conductor in the asset keeps its own state.

Playing conductors from code

You don't have to use an AudioPlayerAsset to play conductors. If you'd rather keep them in your own assets and pick which one to play yourself, call AudioPlayer.PlayConductor() with your own AudioPlaybackSettings - the same settings the AudioPlayerAsset has. Pass the asset holding the conductors as well, so their state and the "Interrupt Same Asset" mode work per asset.

```
[CreateAssetMenu]
public class CharacterSounds : ScriptableObject
{
	public AudioPlaybackSettings Settings;

	public PlayCollectionAudioConductor Footsteps;
	public PlayCollectionAudioConductor FootstepsOnGrass;

	public AudioPlayback PlayFootstep(AudioPlayer player, bool onGrass)
	{
		return player.PlayConductor(Settings, onGrass ? FootstepsOnGrass : Footsteps, this);
	}

	void OnValidate()
	{
		Settings.OnValidate(this);
		Footsteps?.OnValidate(this);
		FootstepsOnGrass?.OnValidate(this);
	}
}
```

Interrupters and tags

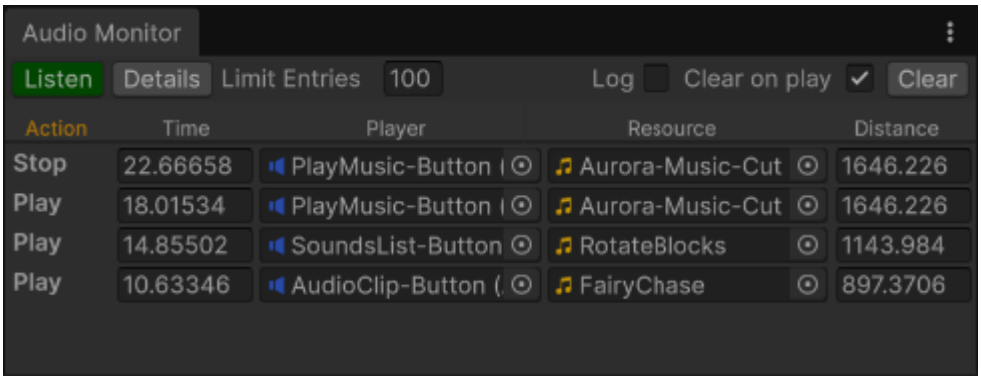
When an asset starts playing (after its delay), its "Interrupter" can stop other sounds that are already playing. For example, a new music track can stop the old one. Leave it empty to not interrupt anything. These interrupters come with the framework:

- StopAllInterrupter - stops all other sounds.
- StopSameAssetInterrupter - stops other sounds started by the same asset, so it doesn't overlap with itself.
- StopByTagsInterrupter - stops sounds whose asset has any of the listed tags.
- StopByMixerInterrupter - stops sounds routed to any of the given output mixer groups or their child groups.
- MultipleInterrupters - runs several interrupters in order.

If you need custom behaviour, extend AudioInterrupter and make your own interrupter type.

Tags are AudioTag assets (create them from "Create / Audio / Audio Tag") that you assign to an AudioPlayerAsset to group its sounds, e.g. "Music", "UI" or "Outdoor".

Audio Monitor



As the project grows you'll need to debug the played sounds at some point. You can use the "Audio Monitor" editor window, found at "Window / Audio / Wise Audio Monitor" in the menu. It shows what sounds were played, when, by whom, distance to the listener, and other details.

Icon made by Magnific from www.flaticon.com